UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA EN CIENCIAS Y SISTEMAS
ORGANIZACIÓN DE LENGUAJES Y COMPILADORES 2
SEGUNDO SEMESTRE 2026
PROYECTO 1

Andy Jefferson González Fuentes 202231338

MANUAL TÉCNICO

1. Introducción y alcance

Este manual técnico documenta la arquitectura interna, las decisiones de diseño y la implementación del compilador desarrollado para el Proyecto 1 del curso de Compiladores 2. Está dirigido a cualquier desarrollador que necesite mantener, extender o evaluar el código fuente del proyecto.

El sistema implementa el frontend completo (análisis léxico, sintáctico y semántico), la generación de código intermedio de tres direcciones (C3D) y dos backends de generación: traducción a código C (Proyecto 1) y traducción a ensamblador RISC-V RV32I con extensión F para flotantes, ejecutable en el emulador RARS (Proyecto 2). Los tres lenguajes soportados son PigLatin, Y y Zetariano (Z). La segunda fase agrega además herencia, polimorfismo, sobrecarga de métodos y dispatch dinámico al lenguaje Zetariano.

A diferencia del manual de usuario, este documento no explica cómo usar los menús de la aplicación, sino cómo está construida internamente: paquetes, clases, flujo de datos entre fases del compilador y las convenciones que debe seguir quien agregue código nuevo.

2. Tecnologías y dependencias

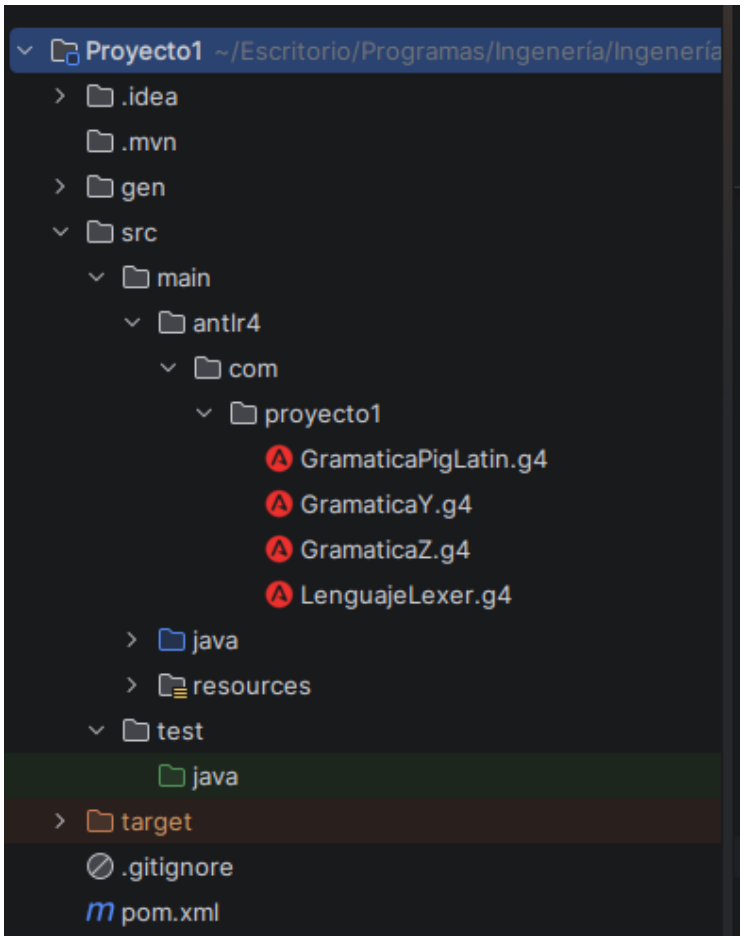
Componente	**Versión / detalle**
Lenguaje	Java 17 (maven.compiler.release=17)
Gestor de build	Apache Maven
Generador de parsers	ANTLR 4.13.1 (antlr4-maven-plugin, con visitor=true)
Interfaz gráfica	JavaFX 21.0.4 (javafx-controls, javafx-fxml)
Empaquetado / ejecución	javafx-maven-plugin 0.0.8, mainClass = com.proyecto1.ui.MainApp
Backend de generación	C (ANSI C), compilable con cualquier compilador estándar (gcc, clang)
Backend de generación (Fase 2)	Ensamblador RISC-V (RV32I + extensión F), target RARS 1.6

El plugin de ANTLR compila las gramáticas ubicadas en src/main/antlr4 y coloca las clases generadas (lexer, parsers y contextos) en target/generated-sources/antlr4, por lo que no se versionan a mano.

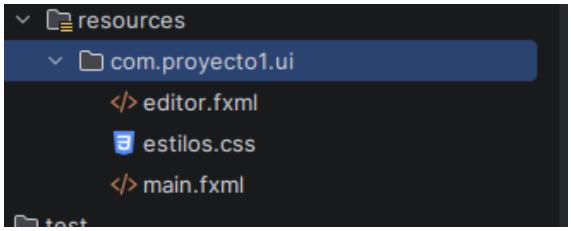
Comando de build y ejecución: "mvn javafx:run" desde la carpeta Proyecto1 (contiene el pom.xml).

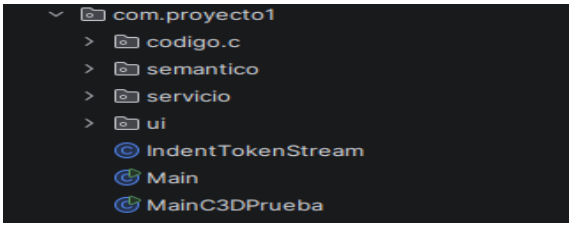
3. Estructura del repositorio

El repositorio contiene un único módulo Maven, Proyecto2, con la siguiente organización:

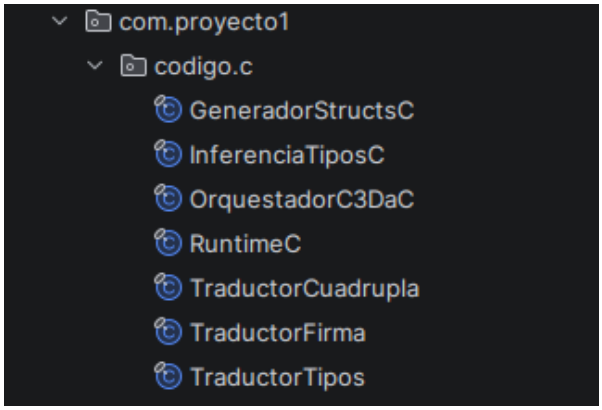


Estilos para las vistas en java fx



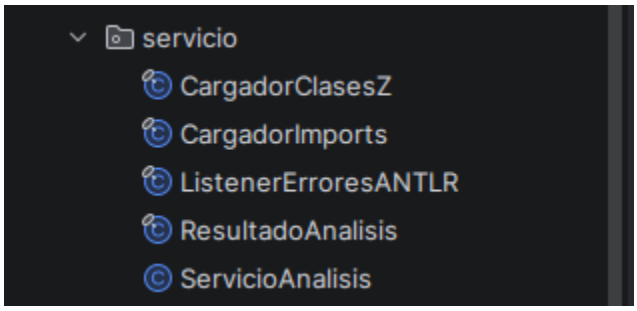


Traducir el código de tres direcciones a c y codigo.riscv:

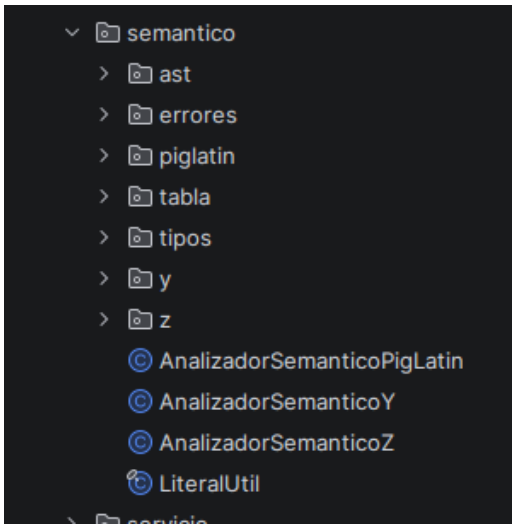


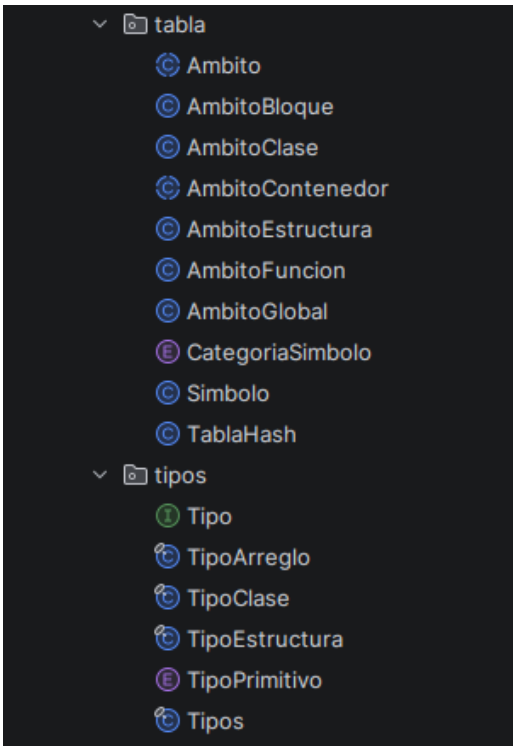


Conexión a la vista



Análisis semántico



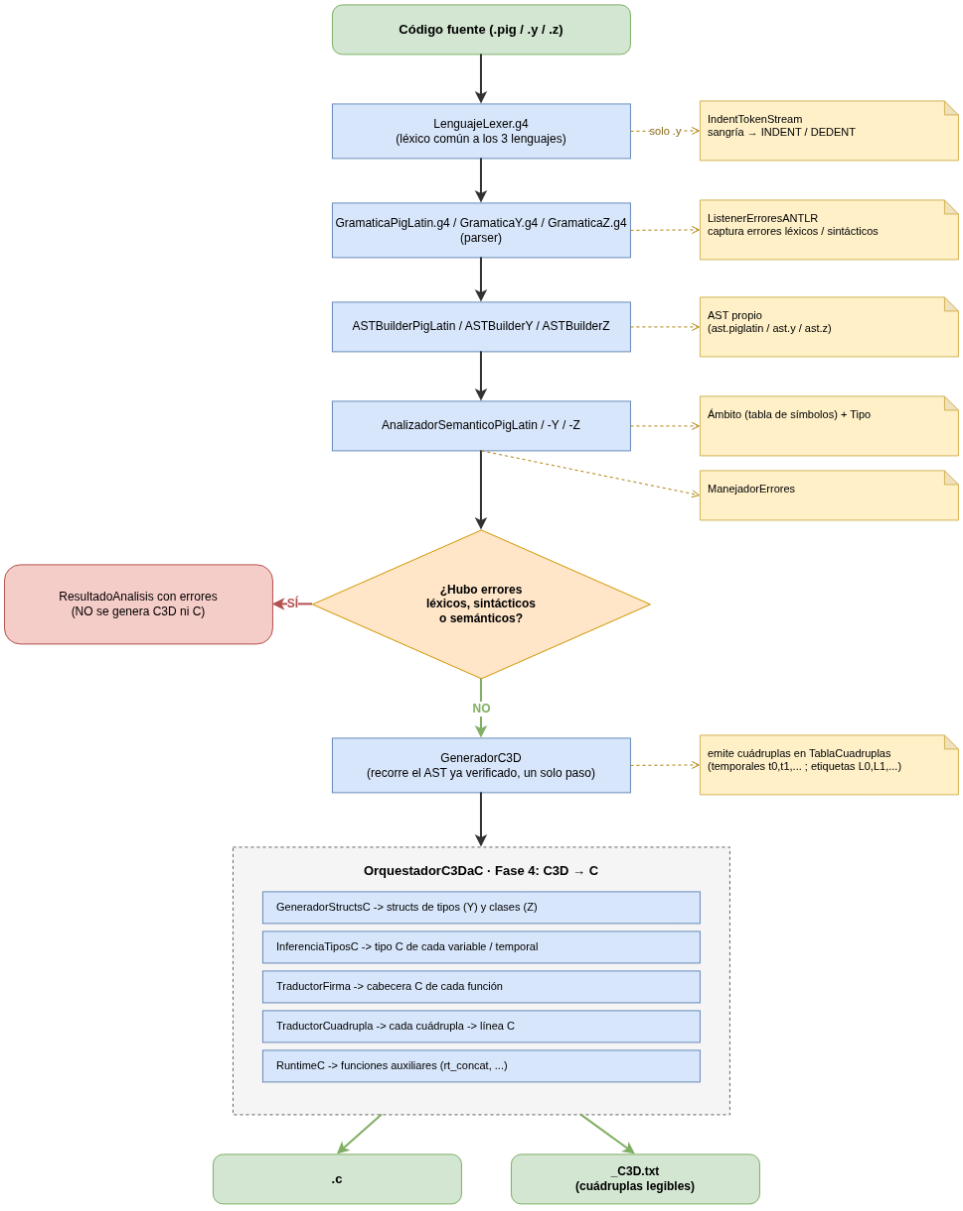


codigo.riscv: Backend RISC-V

4. Pipeline de compilación

Cada archivo (.pig, .y o .z) se procesa a través de una única clase orquestadora, ServicioAnalisis, que encadena las fases del compilador siguiendo un patrón Strategy: un método privado por lenguaje (analizarY, analizarZ, analizarPigLatin), todos con la misma forma. El flujo completo es el siguiente:

Para la Fase 2, después de la generación de C3D, el pipeline continúa con la traducción a RISC-V: las cuádruplas se agrupan por función (begin_func/end_func), se construye el frame de pila de cada una, se traducen las cuádruplas a instrucciones RISC-V y se ensambla el archivo .s completo con secciones .data (literales y heap) y .text (código).



5. Análisis léxico y sintáctico (ANTLR4)

El proyecto usa un único lexer compartido, LenguajeLexer.g4, y tres gramáticas de parser independientes (GramaticaPigLatin.g4, GramaticaY.g4, GramaticaZ.g4), todas con tokenVocab = LenguajeLexer. Cada gramática de parser define su propia regla de entrada (programa para PigLatin y Y, compilationUnit para Z) y usa etiquetas de alternativa (#nombreDef) para que ANTLR genere un contexto y un método de visitor por cada producción, lo cual simplifica el ASTBuilder correspondiente.

5.1 Manejo de la indentación en Y (IndentTokenStream)

El lenguaje Y usa bloques delimitados por sangría, al estilo Python, en vez de llaves. Como ANTLR no soporta indentación de forma nativa, LenguajeLexer.g4 emite un token NEWLINE por cada salto de línea real (dejando espacios/tabs/comentarios en un canal oculto), y la clase IndentTokenStream hace una segunda pasada sobre todo el flujo de tokens ya generado: agrupa los NEWLINE consecutivos, mide el nivel de sangría de la línea siguiente y sintetiza tokens INDENT o DEDENT según aumente o disminuya respecto al nivel anterior. Solo la gramática de Y usa este filtro; PigLatin y Zetariano usan directamente un CommonTokenStream de ANTLR porque delimitan sus bloques con llaves.

5.2 Reporte de errores (ListenerErroresANTLR)

ListenerErroresANTLR reemplaza el listener por defecto de ANTLR (que imprime a stderr en inglés) por dos listas propias en español. Una sola instancia se registra tanto en el lexer como en el parser: se autoclasifica revisando si el "recognizer" que reporta el error es un Lexer (error léxico) o no (error sintáctico). La columna se guarda tal como la entrega ANTLR (getCharPositionInLine(), base 0).

6. Árbol de sintaxis abstracta (AST)

Cada lenguaje tiene su propio paquete de nodos AST (semantico.ast.piglatin, semantico.ast.y, semantico.ast.z), construidos por su respectivo ASTBuilder a partir del árbol de parseo de ANTLR (patrón visitor). Los nodos son independientes entre lenguajes: no hay una jerarquía común de "nodo AST genérico" más allá de exponer el método generarC3D(GeneradorC3D), que cada nodo implementa para emitir sus propias cuádruplas y las de sus hijos, sin guardar estado propio (todo el estado de generación vive en el GeneradorC3D recibido como parámetro).

Este diseño (un ASTBuilder + un paquete de nodos por lenguaje, en vez de un único AST genérico) evita condicionales por lenguaje dispersos en el resto del compilador: el analizador semántico y el generador de C3D de cada lenguaje solo conocen los nodos de su propio paquete.

7. Tabla de símbolos (paquete semantico.tabla)

La tabla de símbolos se implementa como una cadena de Ámbitos (patrón similar a una lista enlazada de tablas hash): para resolver un nombre se busca primero en el ámbito actual y, si no aparece, se sube al ámbito padre, hasta llegar al AmbitoGlobal (el único sin padre). Cada Ámbito usa una TablaHash propia (implementación manual con encadenamiento, sin depender de java.util.HashMap) para sus símbolos locales.

8. Sistema de tipos (paquete semantico.tipos)

La interfaz Tipo define el contrato mínimo (nombre() legible para mensajes de error, y métodos por defecto esNumerico(), esArreglo(), esCompuesto(), esVoid()). Las implementaciones concretas son TipoPrimitivo (enum: entero, flotante, caracter, cadena, bool, void), TipoArreglo, TipoEstructura (tipos definidos por el usuario en Y) y TipoClase (clases de Zetariano).

La clase utilitaria Tipos centraliza las reglas de compatibilidad, en particular esAsignable(destino, origen), usada al validar declaraciones con inicializador, asignaciones, argumentos de llamada y sentencias de retorno. Permite promoción implícita de entero a flotante (no al revés) y trata un tipo desconocido (por un error previo) como compatible con cualquier cosa, para no generar una cascada de errores derivados de uno solo.

9. Manejo de errores semánticos

ManejadorErrores es un recolector simple (una lista de ErrorSemantico) que se instancia una vez por análisis y se pasa explícitamente (no por estático/singleton) a cada verificación en todo el árbol. Esta decisión permite analizar varios archivos en paralelo o en pruebas unitarias sin que se mezclen los errores de uno con los de otro. Cada nodo del AST reporta sus propios errores en el momento en que los detecta y continúa analizando a sus hijos igual, en vez de detenerse en el primer error, de modo que el usuario vea de una vez todos los problemas de su programa.

ErrorSemantico es una clase inmutable con línea, columna y mensaje; se reutiliza tal cual para representar errores léxicos, sintácticos y semánticos (lo que distingue a un error léxico de uno semántico es en qué lista de ResultadoAnalisis cae, no la forma de la clase).

10. Generación de código de tres direcciones (C3D)

GeneradorC3D concentra los servicios que necesita cualquier nodo del AST al traducirse a C3D: pedir un temporal nuevo (t0, t1, ...), pedir una etiqueta nueva (L0, L1, ... para saltos de si/mientras/para), emitir cuádruplas estructuradas, consultar el Ámbito para resolver tipos, construir etiquetas de función/método/constructor (mangling) y registrar la Firma de cada función emitida. Cada nodo recibe este generador como parámetro (programa.generarC3D(generador)); ningún nodo guarda estado propio de generación.

10.1 Cuádruplas y backpatching

TablaCuadruplas es la lista global y ordenada de cuádruplas: la única fuente de verdad del código generado. Un índice (siguienteIndice()) se mantiene estable durante toda la generación, lo que permite backpatching (reemplazar una cuádrupla ya emitida, típicamente un salto cuyo destino aún no se conocía, una vez que ese destino se resuelve). Cuadrupla es una interfaz sellada con una clase concreta por tipo de instrucción, entre ellas:

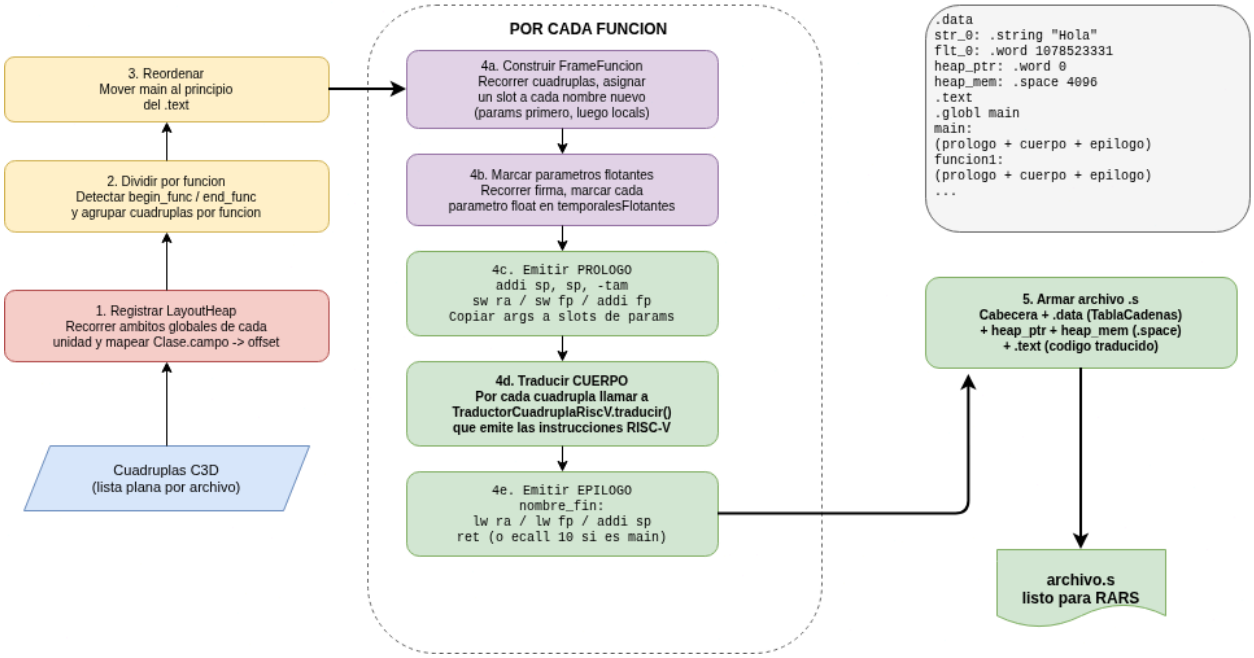
Cuádrupla	**Uso**
CuadruplaBeginFunc / EndFunc	Delimitan el cuerpo de una función/método/constructor
CuadruplaAsignacion	Asignación simple a una variable o temporal
CuadruplaBinaria / Unaria	Operaciones aritméticas, relacionales y lógicas
CuadruplaIfTrue / IfFalse / Goto / Etiqueta / Salto	Control de flujo (si, mientras, para, elegir)
CuadruplaCall / Param / Return	Llamadas a función/método, paso de argumentos y retorno
CuadruplaNew / NewArray	Reserva de memoria para objetos y arreglos (equivalente a malloc)
CuadruplaCampoCarga / CampoGuarda	Lectura/escritura de un campo o atributo (objeto.campo)
CuadruplaIndiceCarga / IndiceGuarda	Lectura/escritura de una posición de arreglo
CuadruplaPrint / Read	Entrada/salida (imprimir/print/println, leer/readln)

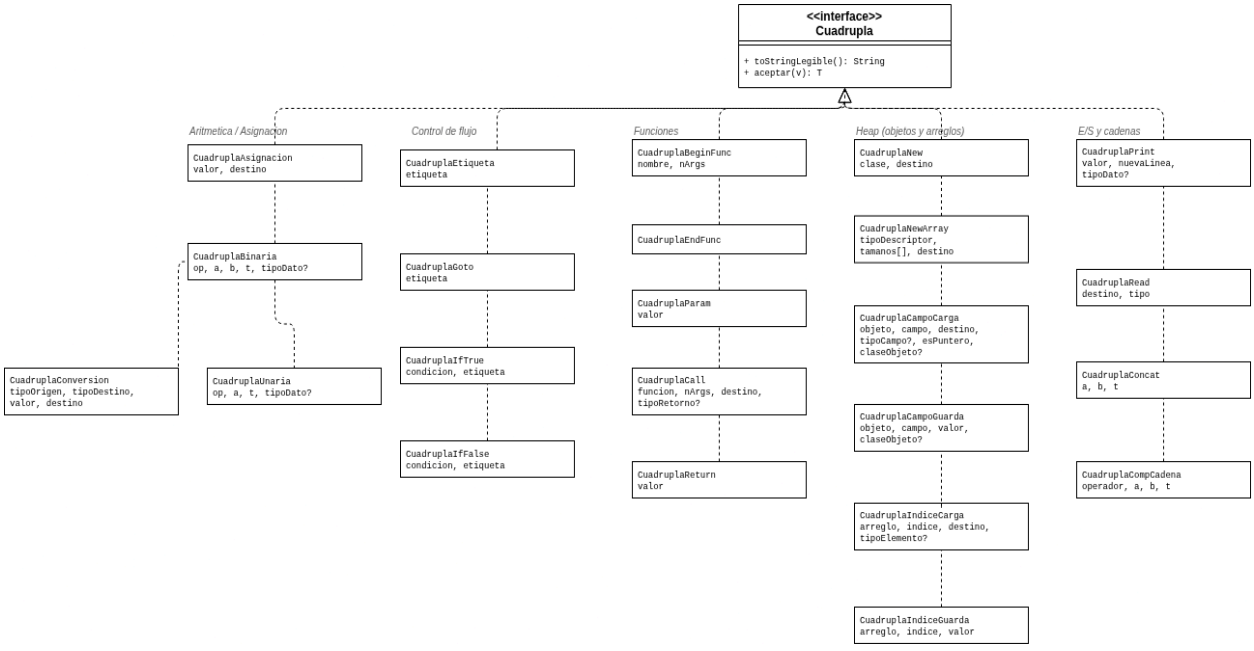
10.2 Mangling de nombres

Para evitar colisiones de nombres en el C generado, cada función/método/constructor recibe una etiqueta única:

- Función suelta de Y: la etiqueta es el nombre tal cual (sin el sufijo interno "#aridad" que usa la tabla de símbolos).
- Método de Zetariano: "Clase_metodo" (por ejemplo, Pila_apilar).
- Constructor de Zetariano: "Clase_init_aN", donde N es la aridad (cantidad de parámetros), construida por GeneradorC3D.etiquetaConstructor(clase, aridad).
- Métodos y constructores reciben siempre "this" como primer parámetro implícito de su Firma; las funciones de Y no.

Pipeline: de Cuadruplas C3D a archivo .s





11. De C3D a C (paquete codigo.c)

OrquestadorC3DaC ensambla el archivo .c completo a partir de las cuádruplas y las Firma de un programa. Sus responsabilidades, delegadas cada una a una clase colaboradora, son:

Clase	**Responsabilidad**
GeneradorStructsC	Emite un typedef struct por cada ESTRUCTURA (Y) o CLASE (Z), en el orden de declaración.
InferenciaTiposC	Infiere el tipo C de cada variable local y temporal de una función a partir de sus cuádruplas y su Firma.
TraductorFirma	Traduce una Firma a la cabecera C de una función (sin llave ni ';', para reusarla como definición o como prototipo).
TraductorCuadrupla	Traduce una cuádrupla a su línea C equivalente, usando el mapa lugar→tipoC de InferenciaTiposC.
TraductorTipos	Traduce un Tipo del lenguaje a su representación en C (las clases y estructuras son siempre punteros, p. ej. Persona*, porque siempre viven en heap).
RuntimeC	Runtime C mínimo inyectado al inicio del archivo generado: rt_read_string, rt_concat, rt_strcmp, rt_int_to_string, rt_double_to_string, entre otras.

Esta separación permite que TraductorCuadrupla decida, por ejemplo, usar rt_concat cuando un '+' involucra un char*, o rt_strcmp cuando '==' / '!=' compara dos char* (en C puro estos operadores no hacen eso), sin mezclar esa lógica con la construcción de la cabecera de la función o de los typedef struct.

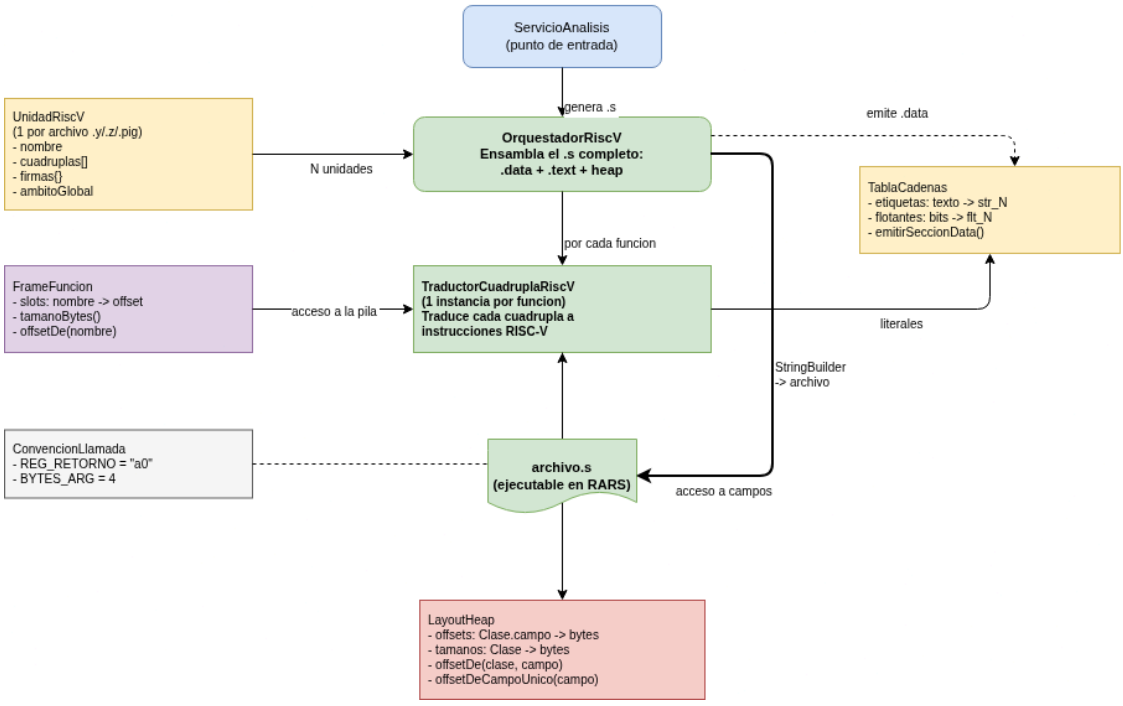
11.1 De C3D a RISC-V (paquete codigo.riscv)

El backend RISC-V reemplaza al backend C del Proyecto 1. En vez de generar un archivo .c, produce un archivo .s de ensamblador RISC-V ejecutable en el emulador RARS. El paquete codigo.riscv contiene siete clases, cada una con una responsabilidad bien definida:

Clase	Responsabilidad
OrquestadorRiscV	Ensambla el archivo .s completo: recibe las unidades de todos los archivos del programa, divide las cuádruplas por función, emite prólogo/epílogo de cada una, y arma las secciones .data y .text

Clase	Responsabilidad
TraductorCuadruplaRiscV	Traduce una cuádrupla individual a instrucciones RISC-V. Existe una instancia por función, con su propio frame, su tabla de flotantes y su contexto de parámetros pendientes
FrameFuncion	Calcula el layout del stack frame de una función: asigna un slot de 4 bytes a cada parámetro, variable local y temporal, y expone offsetDe(nombre) para acceder a cualquier dato como offset negativo de fp
LayoutHeap	Mapea cada campo de cada clase (Z) y estructura (Y?) a su offset en bytes dentro de un objeto del heap. Se construye recorriendo la tabla de símbolos
TablaCadenas	Registra los literales de cadena y flotante del programa y los emite en la sección .data del .s. Un mismo literal repetido se guarda una sola vez
UnidadRiscV	Record inmutable que agrupa el C3D de un archivo fuente (.y, .z o .pig): su nombre, sus cuádruplas, sus firmas y su ámbito global
ConvencionLlamada	Clase utilitaria con las constantes de la convención de llamada: REG_RETORNO = "a0", BYTES_ARG = 4

Arquitectura del Backend RISC-V



11.2 Estrategia general: stack-only, todo por pila

El backend usa una estrategia "stack-only": NADA vive permanentemente en registros entre una cuádrupla y la siguiente. Cada parámetro, variable y temporal tiene su propio slot en la pila, accesible siempre como un offset fijo desde fp (frame pointer). Los registros t0, t1, t2 solo se usan como área de trabajo momentánea para evaluar una operación; el resultado se guarda de vuelta en la pila inmediatamente.

Esta estrategia es más lenta que una con asignación de registros, pero es simple y siempre correcta, incluso con recursión: cada llamada tiene su propio frame, así que un parámetro usado después de un call (como "n" en "n * fact(n-1)") sigue intacto en su slot.

11.3 Convención de llamada

La convención es intencionalmente simple, distinta al ABI oficial de RISC-V:

- Paso de argumentos: TODOS los argumentos se pasan por la pila, siempre, sin importar cuántos sean. El ABI oficial usa a0-a7 para los primeros 8; este compilador no usa esos registros para pasar argumentos.
- Valor de retorno: siempre en a0 (enteros, punteros, booleanos) o fa0 (flotantes).
- Cada argumento ocupa una palabra de 4 bytes en la pila.
- El caller baja sp, apila los argumentos en orden, hace jal, y al volver sube sp para limpiar.
- El callee copia los argumentos de la zona del caller a los slots de su propio frame en el prólogo.

12. Manejo del Stack

12.1 Layout del stack frame (FrameFuncion)

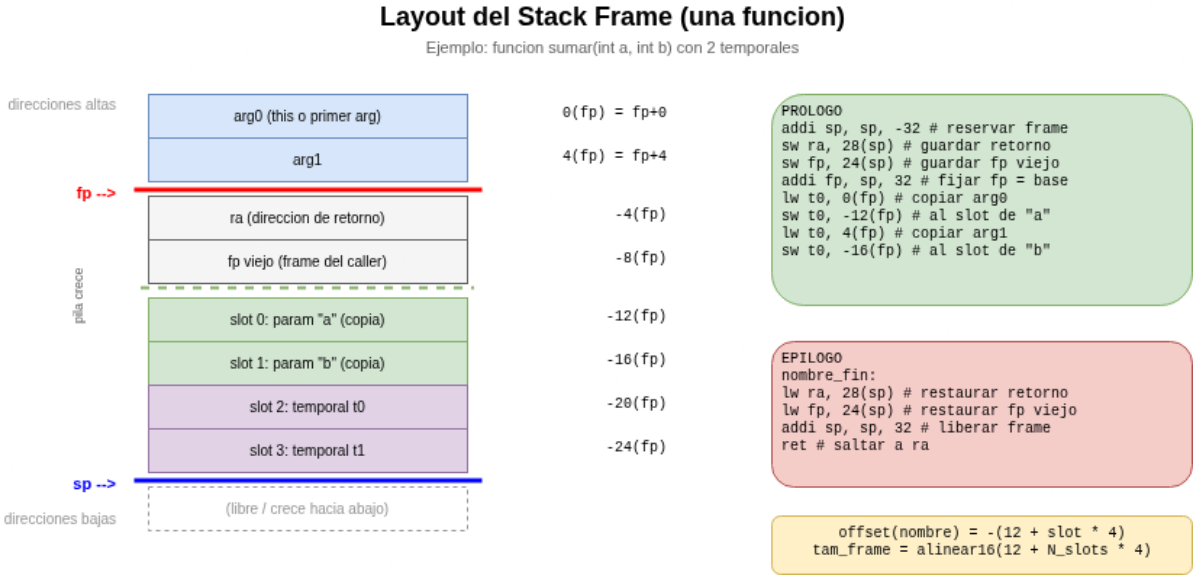
Cada función tiene un frame de pila con el siguiente layout:

Offset desde fp	Contenido
fp + 0, fp + 4, ...	Argumentos del caller (zona de arriba, solo lectura)
fp - 4	ra (dirección de retorno, guardada por el prólogo)
fp - 8	fp del llamador (guardado por el prólogo)
fp - 12	slot 0 (primer parámetro o primer local/temporal)
fp - 16	slot 1
fp - 20	slot 2
...	...

La fórmula es: offset(nombre) = -(12 + slot * 4). Los 12 bytes de cabecera reservan ra y fp viejo; los datos empiezan en fp-12.

FrameFuncion.construir() recorre las cuádruplas del cuerpo de la función y asigna un slot consecutivo a cada nombre nuevo que aparezca (primero los parámetros en orden, luego variables y temporales). Los literales numéricos, cadenas, booleanos y null se filtran por esAlmacenable() y no reciben slot.

El tamaño total del frame se redondea al siguiente múltiplo de 16, como exige el ABI de RISC-V para la alineación de sp.



12.2 Prólogo y epílogo

OrquestadorRiscV.emitirFuncion() genera el prólogo y epílogo de cada función:

Prólogo (al entrar):

1. addi sp, sp, -tam: reservar el frame completo bajando sp.
2. sw ra, tam-4(sp): guardar la dirección de retorno.
3. sw fp, tam-8(sp): guardar la base de frame del caller.
4. addi fp, sp, tam: fijar fp a la base del nuevo frame.
5. Copiar argumentos: para cada parámetro i, lw desde i*4(fp) y sw a offsetDe(param_i)(fp).

Epílogo (al salir, etiqueta nombre_fin):

1. lw ra, tam-4(sp): restaurar ra.
2. lw fp, tam-8(sp): restaurar fp del caller.
3. addi sp, sp, tam: liberar el frame subiendo sp.
4. ret (o ecall 10 si es la función main).

Cada "return" del cuerpo salta a nombre_fin en vez de duplicar el código de salida. El valor de retorno ya viaja en a0 (puesto por CuadruplaReturn).

12.3 Llamadas a función

TraductorCuadruplaRiscV.traducirCall() implementa la secuencia del caller:

1. Acumula los operandos de las cuádruplas CuadruplaParam en una lista (paramsPendientes).
2. Al encontrar CuadruplaCall, baja sp en paramsPendientes.size() * 4.
3. Hace sw de cada argumento en su posición: i*4(sp).
4. Emite jal hacia la etiqueta de la función.
5. Al volver, sube sp para limpiar los argumentos.
6. Mueve el resultado de a0 al slot del temporal destino.

Para llamadas a métodos de Z, el primer argumento es siempre "this" (la referencia al objeto receptor).

13. Manejo del Heap

13.1 Bump allocator

El heap es un bloque fijo de memoria reservado en .data (heap_mem: .space 65536). Un puntero global (heap_ptr: .word) apunta al siguiente byte libre. Cada operación "new" (CuadruplaNew o CuadruplaNewArray) avanza heap_ptr tantos bytes como ocupe el objeto, devuelve la dirección anterior y nunca retrocede. No hay liberación de memoria ni recolector de basura.

La función main inicializa heap_ptr al inicio de heap_mem antes de cualquier otra operación.

Después de cada concatenación de cadenas o conversión a string (i2s, f2s, c2s), el heap_ptr se alinea a 4 bytes (addi + andi con máscara ~3) para que el siguiente lw/sw no falle por desalineación.

13.2 Layout de un objeto de clase Z

LayoutHeap.registrarTipo() construye el mapa de offsets para cada clase:

Offset	Contenido
base + 0	_class_id (entero que identifica la clase real del objeto en tiempo de ejecución; lo usa el dispatch dinámico)
base + 4	Primer atributo (heredado, si la clase tiene padre)
base + 8	Segundo atributo
...	Los atributos propios siguen después de los heredados

Los atributos heredados van PRIMERO y en el MISMO offset que en la clase padre. Esto garantiza que un objeto Perro empieza con exactamente los mismos campos y posiciones que un Animal, permitiendo el polimorfismo: donde se espera un Animal se puede pasar un Perro sin romper los accesos a campos.

El tamaño total del objeto es getTamanoObjeto() * 4 bytes.

[Insertar imagen: diagrama del layout de un objeto Z con herencia]

13.3 Layout de una estructura Y?

Las estructuras de Y? no tienen _class_id (no hay polimorfismo en Y?), así que los campos empiezan directamente en base + 0. Si un campo es un arreglo de tamaño fijo (entero datos[3]), sus elementos se almacenan inline dentro de la estructura y el tamaño se calcula como totalElementos * 4.

13.4 Layout de un arreglo

Un arreglo creado con new int[N] ocupa (N+1) * 4 bytes en el heap:

Offset	Contenido
base + 0	Longitud (N)
base + 4	Elemento [0]
base + 8	Elemento [1]
...	...
base + N*4	Elemento [N-1]

El acceso a un elemento usa la fórmula: base + (indice + 1) * 4. El +1 salta la palabra de longitud. La propiedad .length (AccesoCampo con esLength=true) lee base + 0 usando un índice ficticio de -1: (-1+1)*4 = 0.

Para arreglos bidimensionales (new int[F][C]), se crea un arreglo de punteros de tamaño F, y cada puntero apunta a un arreglo de tamaño C en el heap.

13.5 Cadenas en el heap

Los literales de cadena ("Hola") viven en la sección .data, no en el heap. Las cadenas construidas en tiempo de ejecución (concatenaciones, conversiones i2s/f2s/c2s) sí se crean en el heap como secuencias de bytes terminadas en \0.

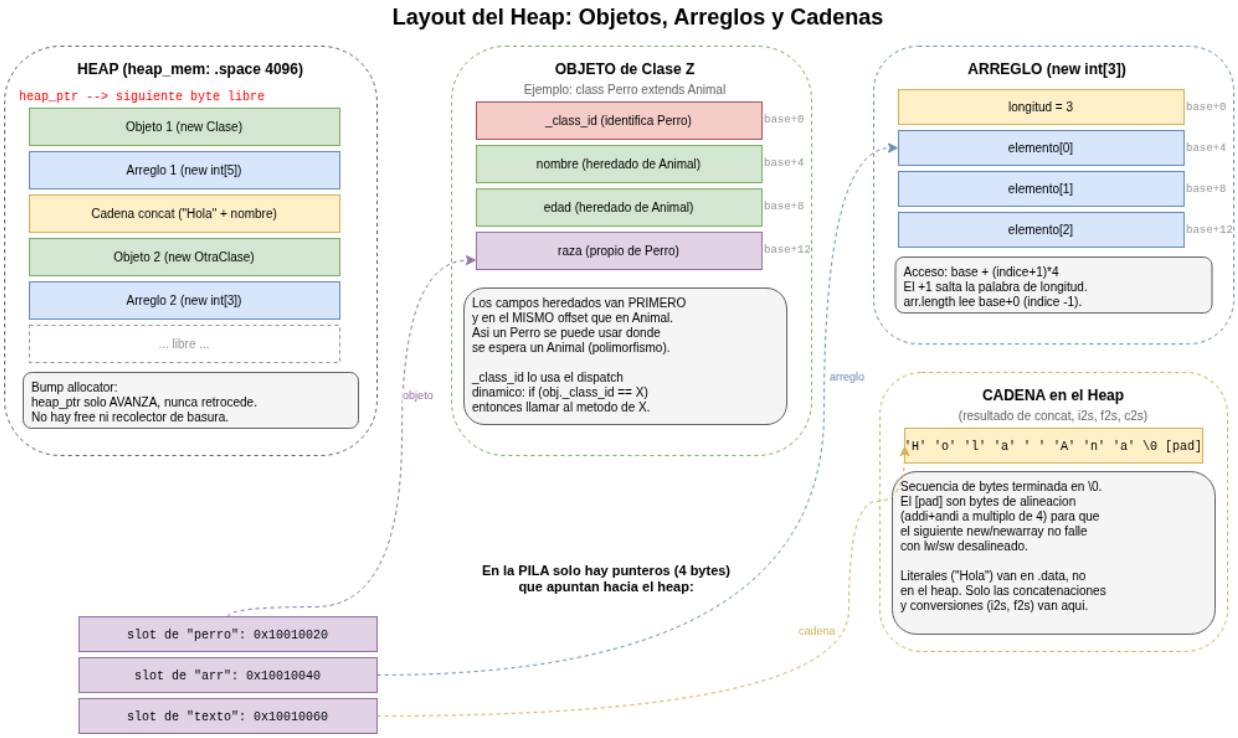
TraductorCuadruplaRiscV.traducirConcat() copia byte a byte las dos cadenas fuente al heap, avanza heap_ptr y alinea a 4 bytes. Las rutinas emitirI2S y emitirF2S convierten un entero o flotante a su representación decimal en ASCII directamente en el heap.

13.6 Resolución de offsets (LayoutHeap)

LayoutHeap ofrece dos formas de resolver el offset de un campo:

- offsetDe(clase, campo): resolución exacta cuando se conoce la clase del objeto. Es el método preferido y se usa siempre que la cuádrupla trae el campo claseObjeto.
- offsetDeCampoUnico(campo): fallback cuando la cuádrupla no dice de qué clase es el objeto. Busca el campo en TODAS las clases; funciona si el nombre es único. Devuelve -2 si es ambiguo (ej. "tamanio" existe en Pila y en Cola con offsets distintos).

En la pila, las variables de tipo objeto, arreglo o cadena guardan solo la DIRECCIÓN (un puntero de 4 bytes) al dato real en el heap. El acceso a un campo se traduce a: lw t1, offset(fp) (cargar puntero del objeto), lw t2, offsetCampo(t1) (cargar el campo).



14. Traducción de cuádruplas a RISC-V (TraductorCuadruplaRiscV)

TraductorCuadruplaRiscV recibe una cuádrupla y un StringBuilder, y emite las instrucciones RISC-V equivalentes. Existe una instancia por función, que conoce su FrameFuncion (para los offsets de la pila), su TablaCadenas (para registrar literales) y su LayoutHeap (para los offsets de campos).

14.1 Carga y guardado de operandos

Cada operando de una cuádrupla puede ser un literal (numérico, cadena, booleano, null) o un nombre (variable, temporal, "this"). El método cargar(operando, registro, sb) despacha según el caso:

Tipo de operando	Instrucción generada
Entero literal ("42")	li t0, 42
Flotante literal ("3.14")	la t0, flt_N / flw ft0, 0(t0)
Cadena literal ("'Hola"')	la t0, str_N
Booleano ("true"/"false")	li t0, 1 o li t0, 0
Null ("null")	li t0, 0
Variable o temporal	lw t0, offset(fp)

El método guardar(registro, destino, sb) hace lo inverso: sw t0, offset(fp).

Para flotantes, existen cargarFloat y guardarFloat que usan flw/fsw y los registros ft0-ft2.

14.2 Detección de operaciones flotantes

El traductor mantiene un Set llamado temporalesFlotantes. Un temporal se marca como flotante cuando: es el destino de una operación con tipoDato="flotante", es el destino de una CuadruplaConversion a flotante, es un parámetro marcado por marcarParametrosFlotantes(), o es el resultado de una CuadruplaCall con tipoRetorno="flotante".

Cuando ambos operandos de una CuadruplaBinaria son flotantes, se emiten instrucciones de la extensión F (fadd.s, fsub.s, fmul.s, fdiv.s, feq.s, flt.s, fle.s) en vez de las enteras (add, sub, mul, div).

14.3 Traducción de cada tipo de cuádrupla

Cuádrupla	Instrucciones RISC-V
CuadruplaAsignacion	cargar fuente en t0, guardar en destino
CuadruplaBinaria (+, -, *, /)	cargar a en t0, b en t1, operar en t2, guardar
CuadruplaBinaria (==, !=, <, >)	cargar, comparar con sub/beqz o feq.s/flt.s, guardar 0 o 1
CuadruplaUnaria (-, !)	cargar, neg/xori, guardar
CuadruplaGoto	j etiqueta
CuadruplaEtiqueta	etiqueta:
CuadruplaIfTrue	cargar condición, bnez -> salto corto, j -> salto largo
CuadruplaIfFalse	cargar condición, beqz -> salto corto, j -> salto largo
CuadruplaParam	acumular en paramsPendientes
CuadruplaCall	bajar sp, apilar args, jal, subir sp, mover a0 a destino
CuadruplaReturn	cargar valor en a0, j nombre_fin
CuadruplaPrint	cargar valor, li a7 servicio, ecall
CuadruplaRead	li a7 servicio, ecall, guardar resultado
CuadruplaNew	calcular tamaño, avanzar heap_ptr, guardar classId, guardar dirección
CuadruplaNewArray	calcular (N+1)*4, avanzar heap_ptr, guardar longitud en palabra 0
CuadruplaCampoCarga	cargar base, lw campo con offset de LayoutHeap
CuadruplaCampoGuarda	cargar base y valor, sw con offset de LayoutHeap
CuadruplaIndiceCarga	cargar base e índice, calcular (idx+1)*4, lw
CuadruplaIndiceGuarda	cargar base, índice y valor, calcular (idx+1)*4, sw

Cuádrupla	Instrucciones RISC-V
CuadruplaConcat	copiar bytes de ambas cadenas al heap, avanzar heap_ptr
CuadruplaConversion	fcvt.s.w (int a float) o fcvt.w.s (float a int con rtz)

14.4 Branches fuera de rango

Los saltos condicionales de RISC-V (beqz, bnez) tienen un rango de +/- 4KB. Para programas grandes que exceden este rango, el traductor usa un patrón de branch invertido + salto incondicional: en vez de "bnez t0, etiqueta_lejana" emite "beqz t0, salto_corto / j etiqueta_lejana / salto_corto:". La instrucción j (JAL con rd=x0) tiene un rango de +/- 1MB, suficiente para cualquier programa del curso.

14.5 Servicios del sistema (ecall)

El código usa los ecalls de RARS con el número de servicio en a7:

a7	Servicio	Argumento	Resultado
1	Imprimir entero	a0 = valor	(nada)
2	Imprimir flotante	fa0 = valor	(nada)
4	Imprimir cadena	a0 = dirección	(nada)
5	Leer entero	(nada)	a0 = valor leído
6	Leer flotante	(nada)	fa0 = valor leído
8	Leer cadena	a0 = buffer, a1 = tamaño	buffer lleno
10	Terminar programa	(nada)	(nada)
11	Imprimir carácter	a0 = código ASCII	(nada)
12	Leer carácter	(nada)	a0 = código ASCII

14.6 Literales en .data (TablaCadenas)

TablaCadenas mantiene dos mapas LinkedHashMap (para conservar el orden de aparición):

- etiquetas: texto decodificado -> etiqueta (str_0, str_1, ...). Se emiten como .string "texto".
- flotantes: bits IEEE 754 (int) -> etiqueta (flt_0, flt_1, ...). Se emiten como .word bits, porque RARS no acepta la directiva .float.

Las cadenas se sanitizan para ASCII puro: las vocales con tilde se reemplazan por la vocal sin tilde, la eñe por n, y cualquier otro carácter no ASCII por '?'.

15. Programas multi-archivo en RISC-V

15.1 UnidadRiscV

RASS ejecuta un solo archivo .s. Cuando se compila un .pig que importa funciones de .y y clases de .z, todas las funciones deben ir en el mismo .s. Cada archivo aporta una UnidadRiscV (record inmutable) con: su nombre sanitizado (ej. "Helpers_y", "Banco_z"), su lista de cuádruplas, su mapa de firmas y su AmbitoGlobal.

El nombre de la unidad se usa como prefijo de las etiquetas de control de flujo (L0, L1...) para evitar colisiones: dos archivos distintos pueden tener ambos un L0, y en un mismo .s chocarían. Con el prefijo quedan "Helpers_y_L0" y "Main_pig_L0".

15.2 OrquestadorRiscV: ensamblado del .s

OrquestadorRiscV recibe la lista de UnidadRiscV y el nombre de la función de entrada ("main" para PigLatin, null para .y/.z sueltos). Su método generarArchivoCompleto() sigue estos pasos:

1. Registrar el layout del heap de todas las unidades (LayoutHeap.registrar para cada AmbitoGlobal).
2. Dividir las cuádruplas de cada unidad por función (detectar begin_func/end_func).
3. Reordenar: la función main va primero en el .text.
4. Traducir cada función (construir FrameFuncion, crear TraductorCuadruplaRiscV, emitir prólogo + cuerpo + epílogo).
5. Armar el archivo: cabecera con comentarios, sección .data (TablaCadenas.emitirSeccionData + heap_ptr + heap_mem), sección .text con todo el código.

La traducción del código (paso 4) se hace ANTES de escribir la sección .data (paso 5), porque es durante la traducción cuando se descubren los literales de cadena y flotante que van en .data.

15.3 Dispatch dinámico (polimorfismo)

Cuando un método puede estar sobreescrito por alguna subclase, GeneradorC3D.dispatchPara() genera una lista de EntradaDispatch: para cada clase que extiende a la clase estática, busca el método en su tabla virtual y produce la etiqueta correcta.

Llamada.emitirDispatch() traduce esto a un if-chain en el C3D:

1. Cargar el _class_id del objeto receptor.
2. Comparar con el classId de cada clase alternativa.
3. Si coincide, saltar al caso de esa clase y llamar a su implementación.
4. Si no coincide con ninguna alternativa, llamar al método de la clase base (caso por defecto).

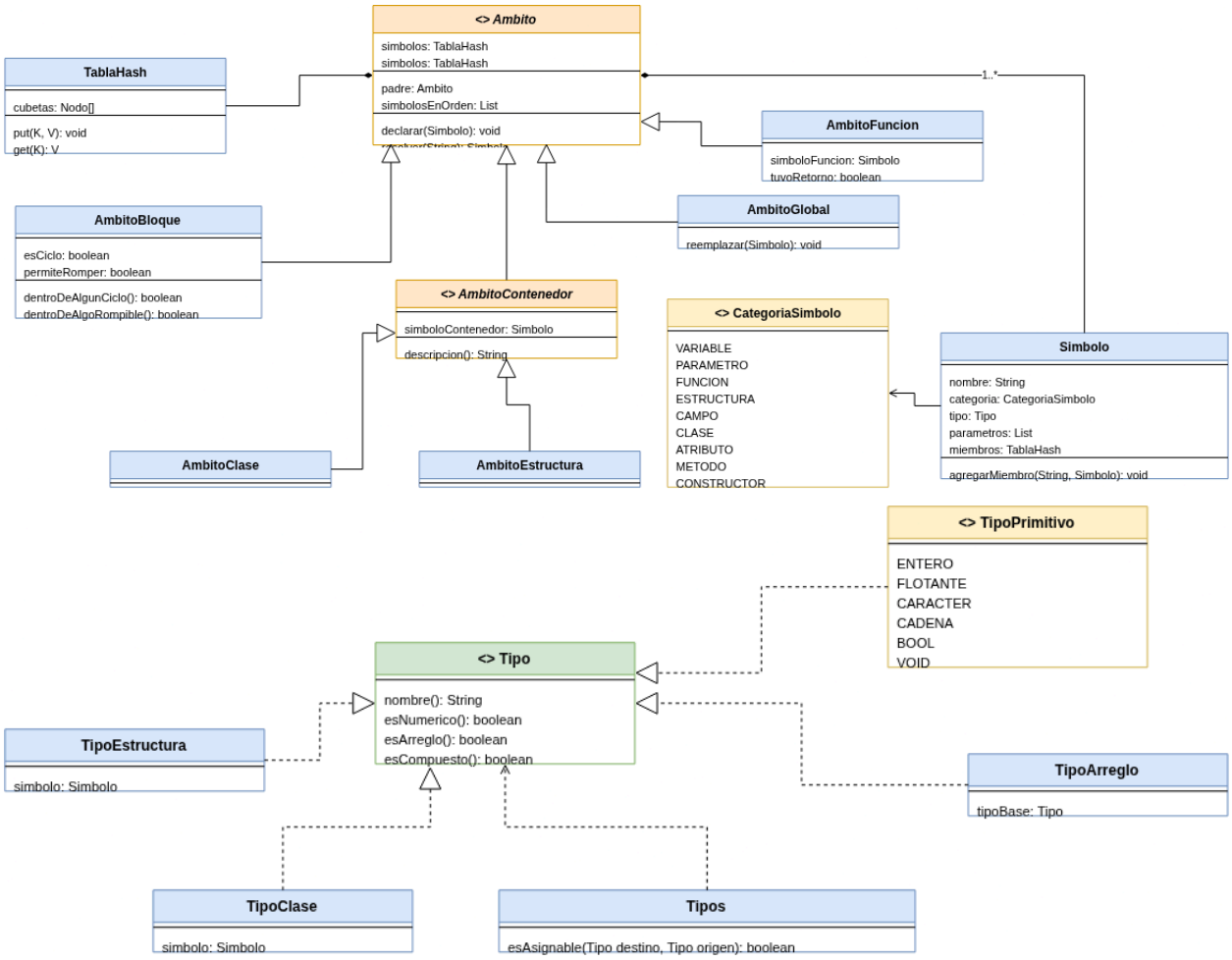
Cada caso dentro del dispatch llama a SU propia etiqueta (ej. Perro_hablar), no a la del default (Animal_hablar).

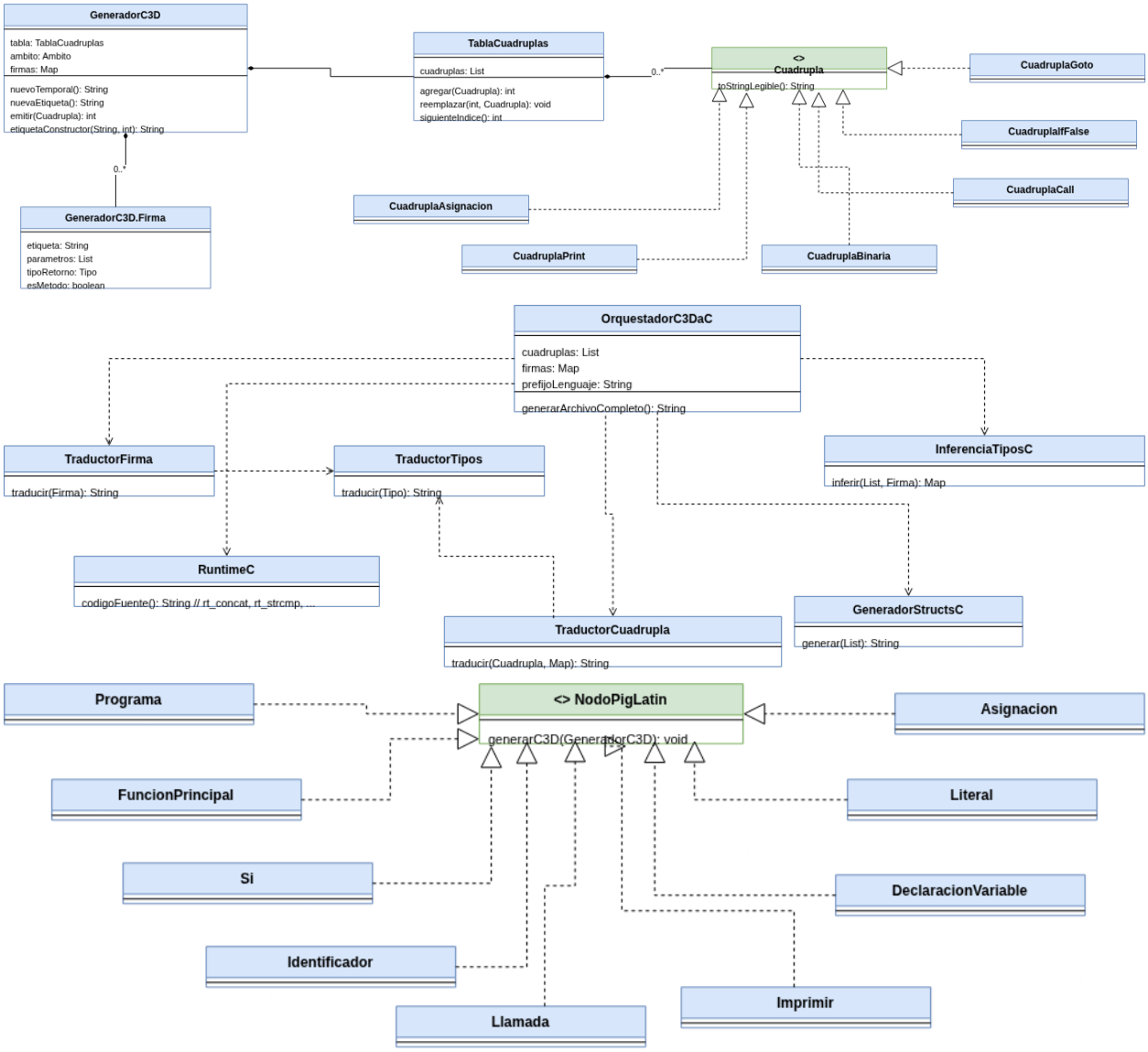
En el .s esto se traduce a una secuencia de lw + beqz/bnez + jal por cada alternativa, seguida del jal al default.

15.4 Interoperabilidad entre lenguajes

PigLatin puede llamar funciones de Y? y métodos de Z en el mismo programa. La interoperabilidad se resuelve en dos niveles:

- Semántico: CargadorImports analiza los archivos importados y registra sus símbolos en el AmbitoGlobal del .pig. ServicioAnalisis.setClasesExternasParaY() inyecta las clases de Z al ámbito de los .y para que puedan usarlas.
- Código: todas las funciones (de Y?, Z y PigLatin) terminan en el mismo .s con etiquetas únicas (gracias al mangling de GeneradorC3D.etiquetaMetodo). Una llamada cross-lenguaje es simplemente un jal a la etiqueta correcta.




GeneradorC3D
tabla: TablaCuadruplas
ambito: Ambito
firmas: Map
nuevoTemporal(): String
nuevaEtiqueta(): String
emitir(Cuadrupla): int
etiquetaConstructor(String, int): String
0..*
GeneradorC3D.Firma
etiqueta: String
parametros: List
tipoRetorno: Tipo
esMetodo: boolean
TablaCuadruplas
cuadruplas: List
agregar(Cuadrupla): int
reemplazar(int, Cuadrupla): void
siguienteIndice(): int
0..*
<>
Cuadrupla
toStringLegible(): String
CuadruplaGoto
CuadruplaIfFalse
CuadruplaCall
CuadruplaAsignacion
CuadruplaPrint
CuadruplaBinaria
OrquestadorC3DaC
cuadruplas: List
firmas: Map
prefijoLenguaje: String
generarArchivoCompleto(): String
TraductorFirma
traducir(Firma): String
TraductorTipos
traducir(Tipo): String
InferenciaTiposC
inferir(List, Firma): Map
RuntimeC
codigoFuente(): String // rt_concat, rt_strcmp, ...
TraductorCuadrupla
traducir(Cuadrupla, Map): String
GeneradorStructsC
generar(List): String
Programa
<> NodoPigLatin
generarC3D(GeneradorC3D): void
Asignacion
FuncionPrincipal
Literal
Si
DeclaracionVariable
Identificador
Imprimir
Llamada

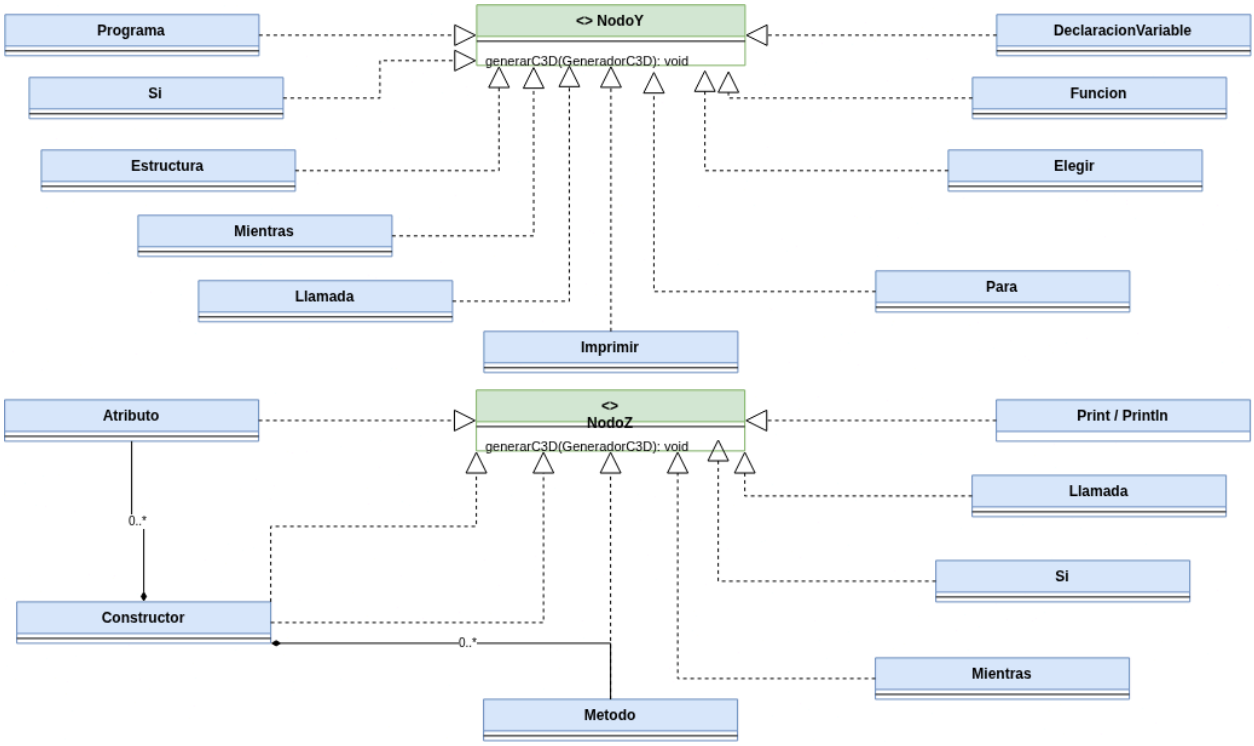


Tabla de Compatibilidad de Tipos

Resultado del operador + entre dos tipos. Los demas operadores aritmeticos (-, *, /, %) solo aplican entre numericos.

Zetariano (Z)

+ (suma/concat)	int	double	char	String	boolean	null
int	int	double	int	String	ERROR	ERROR
double	double	double	double	String	ERROR	ERROR
char	int	double	int	String	ERROR	ERROR
String	String	String	String	String	String	String
boolean	ERROR	ERROR	ERROR	String	ERROR	ERROR

Y?

+ (suma/concat)	entero	flotante	caracter	cadena	bool
entero	entero	flotante	entero	cadena	ERROR
flotante	flotante	flotante	flotante	cadena	ERROR
caracter	entero	flotante	entero	cadena	ERROR
cadena	cadena	cadena	cadena	cadena	cadena
bool	ERROR	ERROR	ERROR	cadena	ERROR

PigLatin

(mismos tipos que Y? con nombres en latin: numerus=entero, decimalis=flotante, littera=caracter, verba=cadena, bool=bool)

+ (suma/concat)	numerus	decimalis	littera	verba	bool
numerus	numerus	decimalis	numerus	verba	ERROR
decimalis	decimalis	decimalis	decimalis	verba	ERROR
littera	numerus	decimalis	numerus	verba	ERROR
verba	verba	verba	verba	verba	verba
bool	ERROR	ERROR	ERROR	verba	ERROR

Leyenda y reglas

- Promocion implicita (int -> float)
- Concatenacion (convierte a string)
- ERROR semantico (tipos incompatibles)

Reglas compartidas por los 3 lenguajes:

1. int + float = float (promocion implicita)
2. string + cualquierTipo = string (concat)
3. char + numerico = int (char como ASCII)
4. bool NO se suma con nada (excepto string)
5. null NO se suma con nada (excepto string en Z)
6. Los operadores -, *, /, % solo entre numericos
7. El operador % es division entera (resultado int)

Asignacion implicita:

- int se asigna a double (promocion, fcvt.s.w)
- double NO se asigna a int (error semantico)
- null se asigna a objetos, arreglos y strings
- Subclase se asigna a clase padre (polimorfismo)

Donde se implementa:

```
- Tipos.resultadoAritmetico(izq, der, esConcat)
- Tipos.esAsignable(destino, origen)
- Binaria.verificar() de cada lenguaje
- TraductorCuadruplaRiscV: emite fadd.s/add
segun temporalesFlotantes
```